MESCAL Specification

Minimalistically Envisioned Smart Contract Assembling Language
Version 1.0.0 — Official Technical Specification

---

1. Introduction

MESCAL (Minimalistically Envisioned Smart Contract Assembling Language) is a JSON-based declarative language designed to define, assemble, and serialize Smart Contracts for KristaTech (KRISTA) compatible blockchains.

Traditional smart contract languages (such as Solidity or Plutus) require specialized developers, heavy runtime virtual machines (VMs), and introduce substantial security attack surfaces. MESCAL solves these issues by adhering to the **KISS (Keep It Simple, Stupid)** principle: it acts as a high-level, human-readable, and machine-parsable JSON representation of Bitcoin-like stack script instructions (`CScript`).

Why MESCAL?

1. **Accessibility**: Allows users and applications to create, sign, and execute smart contracts without needing low-level assembly or compilation knowledge.
2. **Safety**: Operates on a restricted, deterministic instruction set matching blockchain opcodes, eliminating execution-time re-entrancy attacks and gas calculation issues.
3. **Interoperability**: JSON is natively parsed and generated by almost all modern programming languages, making explorer integration, client-side wallets, and backend services trivial.

---

2. Core Philosophy & JSON Formatting

Every MESCAL script is a declarative JSON tree.

- **Order Sensitivity**: Arrays and JSON objects are parsed in strict order of appearance (from the first element to the last).
- **Self-Contained Schemas**: A contract definition compiles directly into linear blockchain bytecode.
- **High-Level Variables**: JSON keys act as parameter bindings, allowing off-chain clients to bind user inputs (such as keys, timestamps, or limits) directly into templates before transaction submission.

---

3. Structural Specification & Component Types

A MESCAL program consists of three fundamental structural types: `basic` , `condition` , and `contract` .

3.1. Basic Elements (`basic`)

A `basic` block represents a primitive script operator or static value wrapper. It maps directly to one or more Bitcoin script opcodes.

```
{
  "type": "basic",
  "name": "Unique-Identifier",
  "role": "<op-code-role>",
  "inputs": [
    {
      "type": "<data-type>",
      "name": "Parameter-Name",
      "value": "<data-value>"
    }
  ]
}
```

Supported Roles & Opcodes

- `hash160`: Performs SHA256 followed by RIPEMD160. Maps to `<val> OP_HASH160` by default, or `OP_HASH160 <20-byte-hash> OP_EQUAL` when `"mode": "htlc-equal"` (or input `"name"` / `"type"` is `"hash160"` with a 20-byte hex value).
- `check-signature-verification`: Verifies a signature against a public key. Maps to `<pubkey> OP_CHECKSIGVERIFY` by default (`"verify": true`), or `<pubkey> OP_CHECKSIG` when `"verify": false`.
- `equalverify-checksig`: Verifies that a public key hash matches the input, then verifies the signature. Maps to `OP_DUP OP_HASH160 <pubkeyhash> OP_EQUALVERIFY OP_CHECKSIG`.
- `number`: Pushes a numerical constant onto the stack. Maps to standard script integers (e.g. `OP_1`, `OP_2` or byte arrays).
- `multi-signature`: Pushes public keys and enforces multisig verification. Maps to `<m> <pubkeys...> <n> OP_CHECKMULTISIG`.
- `lock-time`: Locks transaction spends until a target time or height. Maps to `<locktime> OP_CHECKLOCKTIMEVERIFY OP_DROP`.

3.2. Condition Elements (`condition`)

A `condition` block implements control flow using conditional execution branches (comparisons and boolean evaluations).

```
{
  "type": "condition",
  "name": "Unique-Condition-Name",
  "role": "if-condition",
  "expressions": [
    {
      "required": true,
      "type": "basic",
      "name": "Trigger-Variable"
    }
  ],
  "true": [
    { "type": "basic", "name": "Success-Branch-Action" }
  ],
  "false": [
    { "type": "basic", "name": "Failure-Branch-Action" }
  ]
}
```

- **Script Mapping**: Under the hood, this compiles to `OP_IF ... OP_ELSE ... OP_ENDIF` structures.

3.3. Assembled Contracts (`contract`)

A `contract` is a top-level schema containing sequentially evaluated actions (`basic` or `condition` blocks). It acts as the final script template used to lock transaction outputs (UTXOs).

```
{
  "type": "contract",
  "name": "My-Contract-Name",
  "description": "Human readable documentation for this contract.",
  "actions": [
    { "type": "basic", "name": "Step-1-Action" },
    { "type": "condition", "name": "Step-2-Conditional" }
  ]
}
```

---

4. Complete Specifications & Examples

4.1. HASH160 Evaluation

- **CScript Equivalent**: `<inputs[0]> OP_HASH160`

```
{
  "type": "basic",
  "role": "hash160",
  "name": "Hash-Validation",
  "inputs": [
    {
      "type": "string-or-number",
      "name": "Data-to-Hash",
      "value": "KristaTechSmartContractInput"
    }
  ]
}
```

> **NOTE**
> Additional built-in `basic` roles supported by `CompileBasicBlock` in `src/script/mescal.cpp` include:
>
> - `check-signature-verification`: `<pubkey> OP_CHECKSIGVERIFY` (or `<pubkey> OP_CHECKSIG` when `"verify": false`)
> - `number`: `<int64_t>`
> - `add`, `sub`, `bool-and`, `bool-or`, `num-equal`, `greater-than`, `drop`: `OP_ADD`, `OP_SUB`, `OP_BOOLAND`, `OP_BOOLOR`, `OP_NUMEQUAL`, `OP_GREATERTHAN`, `OP_DROP`
> - `coin-lock-miner`: `<pubkey> OP_DROP <locktime> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_DUP OP_HASH160 <pubkeyhash> OP_EQUALVERIFY OP_CHECKSIG`
> - `pow-miner`: `<nonce> <challenge> <pubkey> OP_DROP OP_DROP OP_DROP <locktime> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_DUP OP_HASH160 <pubkeyhash> OP_EQUALVERIFY OP_CHECKSIG`

4.2. Pay-to-Public-Key-Hash (P2PKH) equivalent

- **CScript Equivalent**: `OP_DUP OP_HASH160 <pubkeyhash> OP_EQUALVERIFY OP_CHECKSIG`

```
{
  "type": "basic",
  "role": "equalverify-checksig",
  "name": "Standard-P2PKH",
  "inputs": [
    {
      "type": "pubkeyhash",
      "name": "Recipient-PubkeyHash",
      "value": "b5a9c9f285d893ce71ab9de8f5c09d765ee982ba"
    }
  ]
}
```

4.3. Multi-Signature (2-of-2)

- **CScript Equivalent**: `2 <pubkey1> <pubkey2> 2 OP_CHECKMULTISIG`

```
{
  "type": "basic",
  "role": "multi-signature",
  "name": "Escrow-Multisig",
  "inputs": [
    {
      "type": "number",
      "name": "n",
      "value": 2
    },
    {
      "type": "number",
      "name": "m",
      "value": 2
    },
    {
      "type": "array",
      "name": "Signatures",
      "value": [
        "02ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660f",
        "03ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660a"
      ]
    }
  ]
}
```

4.4. Time-Locked Deposit (CLTV)

- **CScript Equivalent**: `<expiry-time> OP_CHECKLOCKTIMEVERIFY OP_DROP`


```
{
  "type": "basic",
  "role": "lock-time",
  "name": "Time-Lock-Enforcement",
  "inputs": [
    {
      "type": "timestamp-or-block-height",
      "name": "Lock-Until",
      "value": 1780718400
    }
  ]
}
```

4.5. Conditional Execution (If-Else)

- **CScript Equivalent**: `OP_2 OP_IF <expiry-time> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_1 OP_ELSE <Data-to-Hash> OP_HASH160 OP_ENDIF`

```
{
  "type": "condition",
  "role": "if-condition",
  "name": "Multi-Path-Resolution",
  "expressions": [
    {
      "required": true,
      "type": "basic",
      "name": "Constant-Value-2"
    }
  ],
  "true": [
    {
      "type": "basic",
      "name": "Time-Lock-Enforcement"
    },
    {
      "type": "basic",
      "name": "Constant-Value-1"
    }
  ],
  "false": [
    {
      "type": "basic",
      "name": "Hash-Validation"
    }
  ]
}
```

4.6. Assembled Freezing Funds Contract

Locks funds in a UTXO until a target date. Once the target date passes, the recipient can spend it by providing their signature.

- **CScript Equivalent**: `<expiry-time> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG`


```
{
  "type": "contract",
  "name": "Freezing-Funds",
  "description": "Freezes coins in a UTXO until a future timestamp, then allows spending with
      recipient key.",
  "actions": [
    {
      "type": "basic",
      "name": "Time-Lock-Enforcement"
    },
    {
      "type": "basic",
      "name": "Standard-P2PKH"
    }
  ]
}
```

4.7. Dead Man’s Switch (Inheritance)

If the owner’s key is not used to move funds, after a specific block height or timestamp (expiry), the heir can claim the funds with their signature.

- **CScript Equivalent**: `<heir-pubkey> OP_CHECKSIG OP_IF <expiry> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_1 OP_ELSE <owner-pubkey> OP_CHECKSIG OP_ENDIF`

```
{
  "basic": {
    "Heir-Sig": {
      "role": "check-signature-verification",
      "verify": false,
      "inputs": [
        {
          "name": "Pubkey",
          "type": "pubkey",
          "value": "02ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660f"
        }
      ]
    },
    "Lock-Time": {
      "role": "lock-time",
      "inputs": [
        {
          "name": "Lock-Until",
          "type": "timestamp-or-block-height",
          "value": 1780718400
        }
      ]
    },
    "Push-True": {
      "role": "number",
      "inputs": [
        {
          "name": "True",
          "type": "number",
          "value": 1
        }
      ]
    },
    "Owner-Sig": {
      "role": "check-signature-verification",
      "verify": false,
      "inputs": [
        {
          "name": "Pubkey",
          "type": "pubkey",
          "value": "03ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660a"
        }
      ]
    }
  },
  "condition": {
    "Inheritance-Condition": {
      "role": "if-condition",
      "expressions": [
        { "type": "basic", "name": "Heir-Sig" }
      ],
      "true": [
        { "type": "basic", "name": "Lock-Time" },
        { "type": "basic", "name": "Push-True" }
      ],
      "false": [
        { "type": "basic", "name": "Owner-Sig" }
      ]
    }
  },
  "contract": {
    "Inheritance-Switch": {
```

```
      "description": "Allows heir to spend after lock time passes, otherwise owner can spend
        anytime.",
      "actions": [
        { "type": "condition", "name": "Inheritance-Condition" }
      ]
    }
  },
  "active_contract": "Inheritance-Switch"
}
```

4.8. Dual-Signature Escrow with Mediator (2-of-3)

A standard escrow contract where the Buyer, Seller, and Mediator hold keys. Any 2 out of the 3 parties can sign to release or refund the locked funds.

- **CScript Equivalent**: `2 <buyer-pubkey> <seller-pubkey> <mediator-pubkey> 3 OP_CHECKMULTISIG`

```
{
  "basic": {
    "Escrow-2of3": {
      "role": "multi-signature",
      "inputs": [
        { "name": "m", "type": "number", "value": 2 },
        { "name": "n", "type": "number", "value": 3 },
        {
          "name": "Signatures",
          "type": "array",
          "value": [
            "02ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660f",
            "03ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660a",
            "02cd98ef1234a567bcde0123ef5678cd12345678ab12345678cd12345678ef1234"
          ]
        }
      ]
    }
  },
  "contract": {
    "Escrow-2of3-Contract": {
      "description": "2-of-3 Escrow Contract between Buyer, Seller, and Mediator.",
      "actions": [
        { "type": "basic", "name": "Escrow-2of3" }
      ]
    }
  },
  "active_contract": "Escrow-2of3-Contract"
}
```

4.9. 2-Factor Authentication (2FA) Security Wallet

An operational security policy requiring signatures from both the user's primary mobile wallet and a secondary hardware wallet.

- **CScript Equivalent**: `2 <phone-pubkey> <hardware-pubkey> 2 OP_CHECKMULTISIG`

```
{
  "basic": {
    "Multisig-2of2": {
      "role": "multi-signature",
      "inputs": [
        { "name": "m", "type": "number", "value": 2 },
        { "name": "n", "type": "number", "value": 2 },
        {
          "name": "Signatures",
          "type": "array",
          "value": [
            "02ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660f",
            "03ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660a"
          ]
        }
      ]
    }
  },
  "contract": {
    "Security-Wallet-2FA": {
      "description": "Requires signatures from both primary wallet (mobile) and secondary
        backup (hardware wallet).",
      "actions": [
        { "type": "basic", "name": "Multisig-2of2" }
      ]
    }
  },
  "active_contract": "Security-Wallet-2FA"
}
```

4.10. Hash Time-Locked Swap (HTLC)

A classic atomic cross-chain swap contract. The recipient can claim the funds instantly by providing the secret preimage that hashes to `H`. If the timeout expiry occurs first, the sender can retrieve a full refund.

- **CScript Equivalent**: `OP_HASH160 <hash> OP_EQUAL OP_IF <recipient-pubkey> OP_CHECKSIG OP_ELSE <expiry> OP_CHECKLOCKTIMEVERIFY OP_DROP <sender-pubkey> OP_CHECKSIG OP_ENDIF`

```
{
  "basic": {
    "Preimage-Check": {
      "role": "hash160",
      "mode": "htlc-equal",
      "inputs": [
        {
          "name": "Hash160",
          "type": "string-or-number",
          "value": "b5a9c9f285d893ce71ab9de8f5c09d765ee982ba"
        }
      ]
    },
    "Recipient-Sig": {
      "role": "check-signature-verification",
      "verify": false,
      "inputs": [
        {
          "name": "Pubkey",
          "type": "pubkey",
          "value": "02ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660f"
        }
      ]
    },
    "Timeout-Check": {
      "role": "lock-time",
      "inputs": [
        {
          "name": "Lock-Until",
          "type": "timestamp-or-block-height",
          "value": 1780718400
        }
      ]
    },
    "Sender-Sig": {
      "role": "check-signature-verification",
      "verify": false,
      "inputs": [
        {
          "name": "Pubkey",
          "type": "pubkey",
          "value": "03ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660a"
        }
      ]
    }
  },
  "condition": {
    "HTLC-Branch": {
      "role": "if-condition",
      "expressions": [
        { "type": "basic", "name": "Preimage-Check" }
      ],
      "true": [
        { "type": "basic", "name": "Recipient-Sig" }
      ],
      "false": [
        { "type": "basic", "name": "Timeout-Check" },
        { "type": "basic", "name": "Sender-Sig" }
      ]
    }
  },
  "contract": {
```

```
    "Atomic-Swap-HTLC": {
      "description": "Atomic Swap HTLC: Claimable immediately with secret preimage, or
        refundable to sender after timeout.",
      "actions": [
        { "type": "condition", "name": "HTLC-Branch" }
      ]
    }
  },
  "active_contract": "Atomic-Swap-HTLC"
}
```

4.11. Multi-Path Security Recovery

The owner's signature can spend funds at any time. If the owner's key is lost, a backup recovery team (2-of-3 multisig of trusted friends/services) can recover the funds, but only after a 30-day delay to allow the owner to intercept any malicious recovery attempts.

- **CScript Equivalent**: `2 <friend1> <friend2> <backup> 3 OP_CHECKMULTISIG OP_IF <recovery-delay> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_1 OP_ELSE <owner-pubkey> OP_CHECKSIG OP_ENDIF`

```
{
  "basic": {
    "Owner-Sig": {
      "role": "check-signature-verification",
      "verify": false,
      "inputs": [
        {
          "name": "Pubkey",
          "type": "pubkey",
          "value": "02ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660f"
        }
      ]
    },
    "Recovery-Delay": {
      "role": "lock-time",
      "inputs": [
        {
          "name": "Lock-Until",
          "type": "timestamp-or-block-height",
          "value": 1780718400
        }
      ]
    },
    "Push-True": {
      "role": "number",
      "inputs": [
        {
          "name": "True",
          "type": "number",
          "value": 1
        }
      ]
    },
    "Recovery-Multisig": {
      "role": "multi-signature",
      "inputs": [
        { "name": "m", "type": "number", "value": 2 },
        { "name": "n", "type": "number", "value": 3 },
        {
          "name": "Signatures",
          "type": "array",
          "value": [
            "03ee1fb80068f574b0d110009f110f703161cd358889a7bc48c613aa898136660a",
            "02cd98ef1234a567bcde0123ef5678cd12345678ab12345678cd12345678ef1234",
            "03ab89ef1234a567bcde0123ef5678cd12345678ab12345678cd12345678ef1235"
          ]
        }
      ]
    }
  },
  "condition": {
    "Recovery-Path": {
      "role": "if-condition",
      "expressions": [
        { "type": "basic", "name": "Recovery-Multisig" }
      ],
      "true": [
        { "type": "basic", "name": "Recovery-Delay" },
        { "type": "basic", "name": "Push-True" }
      ],
      "false": [
        { "type": "basic", "name": "Owner-Sig" }
```

```
      ]
    }
  },
  "contract": {
    "Multi-Path-Recovery": {
      "description": "Owner can spend anytime. Recovery team (2-of-3 multisig) can recover
        funds only after a recovery lock delay.",
      "actions": [
        { "type": "condition", "name": "Recovery-Path" }
      ]
    }
  },
  "active_contract": "Multi-Path-Recovery"
}
```

---

5. Compiler Implementation Guidelines

To deploy MESCAL contracts onto KristaTech (KRISTA) nodes, a parser must process the JSON tree and output the serializable `CScript` bytes.

5.1. Parsing Order

The compiler must parse the top-level `"actions"` array sequentially:

1. Load action definitions recursively.
2. Resolve referenced parameters and inject variables.
3. Emit matching opcode equivalents onto the byte compilation stream.

5.2. Validation Rules

Before emitting CScript bytecode, the compiler must assert the following safety checks:

- **Stack Depth Limit**: Verify that the generated opcodes do not exceed the blockchain's maximum stack depth limit (usually 1000 items).
- **Disabled Opcodes**: Prevent injection of prohibited opcodes (e.g. `OP_CAT` or `OP_LSHIFT`) which are disabled on the consensus layer to prevent memory exhaustion.
- **Time Lock Assertions**: Ensure that any `lock-time` actions place the locktime opcode (`OP_CHECKLOCKTIMEVERIFY`) before signature checking to avoid transaction malleability issues.

---

6. Taproot (P2TR) Script-Path Integration

MESCAL contracts can be compiled directly into Taproot (P2TR) script-path spending conditions. This allows developers to hide contract scripts inside a Taproot output, revealing them only during a script-path spend.

6.1. The `compilemescaltotaproot` RPC Command

The daemon provides a built-in RPC command `compilemescaltotaproot` to compile a MESCAL contract and generate the required Taproot parameters.

Command Arguments

1. `json` (string, required): The MESCAL contract JSON string.
2. `internal_pubkey` (string, required): A 32-byte (64 character) hex-encoded x-only public key representing the key-path spend master key.

Command Example

```
kristatech-cli compilemescaltotaproot '{"basic": {"MyDrop": {"role": "drop"}}, "contract":
        {"TestDrop": {"actions": [{"type": "basic", "name": "MyDrop"}]}}, "active_contract":
        "TestDrop"}' "697caf5a1ea29fa2e41e32ce0514e6c11d0b549be82136369ce8d1d435e9e731"
```

Command Result Object

- `address` (string): The Bech32m-encoded Taproot (P2TR) address.
- `scriptPubKey` (string): The locking scriptPubKey for the transaction output.
- `leafScript` (string): The compiled MESCAL script hex.
- `controlBlock` (string): The witness control block (33 bytes hex) proving leaf script inclusion in the Taproot commitment.

6.2. Spending P2TR Script-Path UTXOs

To spend a UTXO locked at a Taproot address via the script-path (using the compiled MESCAL contract):

1. **Create the raw spending transaction**: Inputs must spend the funded P2TR output.
2. **Inject the scriptSig witness parameters**: In the spending input's `scriptSig`, construct the byte array: `[witness_stack_items...] + [leafScript] + [controlBlock]`

   Note: In the KristaTech VM interpreter, the scriptSig for a Taproot script-path spend receives the contract execution arguments first, followed by the leaf script push, followed by the control block push. Inside `SIGVERSION_TAPSCRIPT` leaf scripts, `OP_CHECKSIG` / `OP_CHECKSIGVERIFY` / `OP_CHECKMULTISIG` support both 32-byte x-only BIP340 Schnorr public keys and standard 33/65-byte MESCAL ECDSA public keys (automatically routed to `SIGVERSION_BASE` DER ECDSA verification).

Spending ScriptSig Construction Example (Python)

```python
# witness_stack_items: two OP_1 items (0x5151) so MyDrop (OP_DROP) pops one and leaves OP_1
        (true) on the stack
witness_stack = bytes.fromhex("5151")
# leafScript: OP_DROP (0x75) compiled from the MyDrop action in Section 6.1
leaf_script = bytes.fromhex("75")
# controlBlock: e.g. 33-byte control block returned by RPC
control_block = bytes.fromhex(control_block_hex)

# Construct final scriptSig:
# Pushes the witness parameters, leaf script, and control block
script_sig = witness_stack + bytes([len(leaf_script)]) + leaf_script +
        bytes([len(control_block)]) + control_block
```